Animate Subject Tracks

Technical Guide

Animated 3D Movement Maps & Video — Pipeline & Rendering Reference

Workflow version 14.0.0 · Generated November 06, 2026

1. Overview

The **Animate Subject Tracks** workflow turns the GPS history of an EarthRanger subject group into an animated, terrain-aware 3D map. Each subject's track is cleaned into trajectory segments, stitched into one timestamped path per subject, draped onto a digital elevation model, and played back on a shared clock over a basemap the user chooses (satellite imagery by default).

The workflow produces an interactive HTML map (deck.gl, via pydeck) with a playback bar and a Camera panel, GeoParquet copies of the relocations and trajectories, and a dashboard with the map and a Files section. A single **Camera** step sets the camera that the interactive map previews while playing and that the video follows. The workflow can optionally record an MP4 video frame by frame from that map in a headless browser; video export is off by default.

> **Note:** The workflow has no groupers: `set_groupers` is configured with an empty list, so every subject in the group is animated together on one map.

1a. Task Order

The tasks run in the order below (task ids in the left column). Tasks in a task group appear on the configuration form under the group's title; fixed values set in the spec are not on the form.

Form section	Task ids, in spec order
Setup	`workflow_details`, `er_client_name`, `time_range`, `groupers`
Data ingestion	`subject_observations`, `subject_reloc`, `convert_to_trajs`, `rename_traj_cols`, `persist_relocs_geoparquet`, `persist_trajs_geoparquet`
Terrain (task group)	`terrain_exaggeration`, `terrain_sampling`, `terrain_layer`
Animation data	`ensure_wgs`, `trajs_trips`, `drape_terrain`
Map Zoom & Extent (task group)	`zoom_to_envelope`, `trips_view_state`
Colours	`rgba_hex`
Animation (task group)	`trips_animation`, `create_controls`, `timeline_animation`, `video_camera`
Map	`trips_layer`, `animate_trips`, `draw_animation`, `map_urls`
Video Export (task group)	`export_video_toggle`, `create_animation`
Dashboard	`animated_map_widget`, `map_files`, `video_files`, `animate_dashboard`

> **Note:** The earlier **Configure Basemap URLs** (`custom_basemap_urls`) and **Configure basemap** (`basemap_option`) steps have been removed. The basemap is now chosen on the Terrain group's **Basemap** step (section 4b).

2. Dependencies & Prerequisites

2a. EarthRanger

Observations are pulled live from EarthRanger through the connection chosen in **Connect to EarthRanger** (`set_er_connection`). The user picks a **Subject Group Name** from a drop-down of the chosen data source's subject groups; every subject in that group is included. A group with mixed subtypes can give unexpected results.

2b. External Tile Services & Map Widgets

The 3D terrain, its texture and the map's widgets are fetched at run time from public services:

Service	URL template	Used for
AWS Terrarium elevation	`https://s3.amazonaws.com/elevation-tiles-prod/terrarium/{z}/{x}/{y}.png`	Terrain mesh and sampling ground height for draping
Esri basemap (user's choice)	`https://server.arcgisonline.com/ArcGIS/rest/services/<service>/MapServer/tile/{z}/{y}/{x}`	Texture draped on the terrain mesh. One of the four basemaps in section 4b; World Imagery (Satellite) by default
Esri World Boundaries and Places	Esri tile service (same tile scheme)	Optional place names, draped on the terrain (**Place names**, off by default)
my-deck-widgets (jsDelivr)	`https://cdn.jsdelivr.net/gh/ttemu34/my-deck-widgets@v0.21.0/dist/bundle.js`	The map's widgets: playback bar, Camera panel, legend, north arrow, scale bar. Loaded by ecoscope-workflows-ext-mep

All are needed by the interactive map and, when enabled, by the video renderer, so the machine running the workflow (and anyone viewing the HTML map) needs internet access.

2c. Headless Browser

Video rendering, which only runs when **Export video** is on (section 7), uses Playwright's Chromium. The renderer uses a Chromium bundled in the environment when there is one; if the browser executable is missing it runs `playwright install chromium` once and retries. Encoding uses the ffmpeg binary bundled with `imageio-ffmpeg`.

3. Data Ingestion Pipeline

Step (task id)	Task	What it does
`subject_observations`	`get_subjectgroup_observations`	Fetches observations for the subject group over the time range, with `filter: clean` (only fixes with exclusion flag 0), observation details and subject-source details. `raise_on_empty: false` returns an empty frame instead of failing.
`subject_reloc`	`process_relocations`	Converts observations to relocations, keeps the subject name, colour, sex, subtype and source columns, and drops fixes at the known junk coordinates (180, 90), (0, 0) and (1, 1).
`convert_to_trajs`	`relocations_to_trajectory`	Builds trajectory segments between consecutive fixes and applies the user's **Trajectory Segment Filter**.
`rename_traj_cols`	`map_columns`	Renames `extra__hex` → `hex_color`, `extra__name` → `subject_name`, and the sex, subtype and created-at columns. Missing columns are ignored.
`persist_relocs_geoparquet`	`persist_df_wrapper`	Writes relocations to `relocations_<hash>.geoparquet`, sanitising nested objects for Arrow.

Step (task id)	Task	What it does
`persist_trajs_geoparquet`	`persist_df_wrapper`	Writes trajectories to `trajectories_<hash>.geoparquet`.

3a. Trajectory Segment Filter Defaults

Limit	Minimum	Maximum
Segment length	0.001 m	100 000 m
Segment duration	1 s	172 800 s (2 days)
Segment speed	0.01 km/h	500 km/h

4. Terrain

On the configuration form, `terrain_exaggeration`, `terrain_sampling` and `terrain_layer` are grouped in a single **Terrain** task group, shown as the **Terrain Exaggeration**, **Terrain Sampling** and **Basemap** sub-sections.

4a. Elevation Decoder & Exaggeration

`create_elevation_decoder` builds the RGB-to-elevation decoder for Terrarium tiles (`r_scaler 256`, `g_scaler 1.0`, `b_scaler 1/256`, `offset -32768`). deck.gl's TerrainLayer has no elevation-scale property, so **Exaggeration** (default 1.0; 2.0 doubles heights) is applied by multiplying every decoder term. The same decoder feeds both the terrain mesh and the track sampling, so the tracks always sit on the exaggerated surface.

4b. Terrain Layer & Basemap

`create_terrain_layer` defines the 3D terrain mesh from the Terrarium tiles, zoom range 0–15 and wireframe off. Its `texture` is left open on the form as the **Basemap** field. The spec's `rjsf-overrides` turn it into a drop-down (a `oneOf` list) of four Esri tile services, so users choose by name rather than typing a URL. The video renderer records the same page, so it uses the same basemap.

Option	Esri service	Notes
Satellite	`World_Imagery`	Default
Terrain	`World_Topo_Map`	Topographic map with contours and labels
Roadmap	`World_Street_Map`	Street map
USGS Hillshade	`Elevation/World_Hillshade`	Grey shaded relief

Place names (off by default) adds the Esri World Boundaries and Places layer over the chosen basemap, draped on the terrain like the imagery. The spec also fixes the following values, which are not on the form:

Setting	Value
Imagery detail	`imagery_max_zoom 18`: imagery is draped at up to zoom 18 (about 0.6 m a pixel) over the zoom-15 terrain. Where the basemap has nothing that deep, the terrain's own imagery shows
Imagery look	`natural`: slightly lifted contrast and colour, a little warmer. Applies to the ground only; tracks and markers keep their legend colours
Vignette	`0.25`: soft dark edges on the map and in the video

Setting	Value
Terrain detail	`mesh_max_error 1`: the 3D terrain may stray up to 1 m from the elevation data

4c. Draping Tracks on Terrain

`create_terrain_sampling` configures how track heights are set; `drape_trips_on_terrain` then applies it, setting the z value of every track vertex. The user picks a **Track height** using the Google Maps 3D altitude modes. Each mode shows only its own field:

Track height	Mode	Field and effect
On the ground	`CLAMP_TO_GROUND`	No field. Tracks follow the terrain; recorded altitudes are ignored.
Above the ground (default)	`RELATIVE_TO_GROUND`	**Height (m)**, default 30: sampled ground height plus the height.
Recorded altitude (where available)	`ABSOLUTE`	**Height without an altitude (m)**, default 30: each fix's own altitude is used. Fixes with none (usual for EarthRanger data) are drawn that height above the ground, not at sea level.
Above the 3D terrain	`RELATIVE_TO_MESH`	**Height (m)**, default 30: tracks, markers and labels are drawn that height above the terrain as rendered, so they stay on the surface at every zoom.

The spec sets `densify_m: 150`, which is not on the form: before heights are set, every track gets a point every 150 m, with timestamps interpolated, so lines follow the terrain between fixes instead of cutting through hills. Steps across data gaps are left alone. If the total would be too large, the spacing widens automatically. All vertices are sampled in one batched call, so each elevation tile is fetched once. If the elevation tiles cannot be read, every vertex uses a constant 1000 m ground (an internal fallback, not on the form).

5. Animation Data

Step (task id)	Task	What it does
`ensure_wgs`	`convert_crs`	Reprojects trajectories to EPSG:4326.
`trajs_trips`	`trajectory_to_trips`	Stitches each subject's segments (grouped by `groupby_col`) into one lon/lat LineString with per-vertex timestamps, keeping `subject_name` and `hex_color`. Breaks of 12 hours or more split a track, and pieces with fewer than 5 fixes are dropped.
`drape_terrain`	`drape_trips_on_terrain`	Densifies tracks and sets terrain heights on every vertex (section 4c).
`zoom_to_envelope`	`envelope_gdf`	Takes the bounding envelope of all trips (expansion factor 1.0).
`trips_view_state`	`compute_view_state_from_gdf`	Computes the centre and zoom that fit the envelope (max zoom 15), plus the user's **Pitch** (default 45°, 0–90) and **Bearing** (default 0°, –180 to 180). The map opens framed on all tracks, and the video cameras use this tilt and heading too.

Step (task id)	Task	What it does
`rgba_hex`	`add_rgba_from_hex`	Converts each subject's EarthRanger hex colour to an RGBA `rgba_color` column for the trail and legend.

6. Animated Map

On the configuration form, `trips_animation`, `create_controls`, `timeline_animation` and `video_camera` are grouped in a single **Animation** task group, shown as the **Create Animation**, **Playback Control**, **Playback Length & Smoothness** and **Camera** sub-sections.

6a. Trips Animation

`create_trips_animation` defines how each track plays back. Each subject shows a bright comet tail in its own colour covering 95% of the timeline (`comet_ratio 0.95`), over a faded white history line of the whole path travelled so far (opacity 0.85). Turns are smoothed, and there is no hover tooltip and no marker label. The user picks the **Marker icon** drawn at each subject's current position, either a dot or a 3D animal:

Marker	Rendering
Dot (default)	ScatterplotLayer circle, 4 px radius, subject colour, 1.5 px white outline
3D animal	Preset ScenegraphLayer model: elephant, giraffe, cheetah, leopard or lion. **Size**, **Use subject color** and **Realistic lighting** are adjustable

6b. Trips Layer

`create_trips_layer` draws the draped trips as a deck.gl TripsLayer: `timestamps` as the time accessor, `rgba_color` as the colour, 2.15 px width (clamped to 1–4 px), rounded caps and joints. The legend is titled **Subjects**, labelled by `subject_name` and sorted ascending. `animate_layer` attaches the trips animation to this layer.

6c. Playback Controls & Timeline

`create_playback_controls` configures the playback bar. The user can set its **Position** (bottom or top; default bottom), whether it shows the **Camera** button (default on) and whether the map buttons are shown (**Show map buttons**, default on). The spec fixes everything else:

- Play/pause, restart, time slider, clock (playback position over total length, for example 0:12 / 0:30) and the data date-time.
- Speed button cycling 0.25×, 0.5×, 0.75×, 1×, 2× and 4×.
- **Preview camera while playing** is on, and the how-to-move hint is off.

`create_timeline_animation` builds the shared clock. The user sets the playback length **Duration** (default 30 s): the whole timeline plays in that time, and the video uses it by default. The spec fixes the redraw rate at 60 fps (`fps_limit`), so it is no longer a form field. `auto_rotate_speed` is 0, so the map does not rotate while it plays.

6d. Camera Step & Camera Panel

`set_video_camera` (task id `video_camera`, form step **Camera**) sets one camera for both the interactive map and the video. Its output is passed to `draw_animated_map` (so the map's Camera panel opens on it and previews it while playing) and to `render_video_from_settings`, so the map and the video always agree. The spec restricts the **Camera settings** field to a single option, **From Camera panel settings**: the settings copied from the map's Camera panel, pasted into the field, or the path of the downloaded `.json` file. The default is `{"type_": "fixed"}`, a fixed view of the whole scene. The settings are validated when the step runs, so a missing or bad waypoint file fails there rather than part way through a video.

The workflow is therefore: run it, open the interactive map, design a camera in the Camera panel, copy its settings, paste them into the workflow's **Camera** step and re-run. The Camera panel (opened from the playback bar's camera button) lets the user:

- Pick a camera and set its fields (list in section 6e).
- Press **Use current view** to fill the centre, range, tilt and heading from the map as it is now.
- Turn **Preview camera while playing** on (the default) or off. Moving the map by hand turns it off, with a notice.
- **Copy settings** or **Download** the camera, to paste into the workflow's Camera step. The panel opens on the workflow's camera and floats beside the top-right map buttons.

6e. Camera Options

Cameras are described the Google Maps 3D way: a centre the camera looks at, its range (distance in metres), tilt, heading and field of view. Blank values frame the whole scene at the map's tilt and heading.

Camera	Behaviour	Main settings
Fixed view	One view; blank centre = the whole scene	Centre, Range, Tilt, Heading, Field of view
Fly to & around	Flies in from the whole scene, then circles the centre	Centre, Range, Tilt, Heading, Fly Frac 0.25 of the clip, Rounds 1
Follow a subject	Sits behind one subject, facing its travel	Subject (blank = longest-running track), Range 800 m, Tilt 60
Subject tour	Follows each subject in turn for an equal share of the clip, flying between them	Subjects list (the order; blank = all by name), Fly between 0.2, Name cards (3 s as each subject comes in; on)
Follow the action	Frames the recent movement of all (or listed) subjects	Subjects, optional Face travel
Cinematic fly-through	Whole-scene intro, then a tilted, turning follow of the action	Fly-in 0.12, Tilt 55, Turning (rotate / face heading / fixed), Turn 45°
Keyframes	Flies through waypoints added in the panel, uploaded from a file, or derived from a subject's track	Source (waypoints / file / subject), easing smooth / linear / spline

The four following cameras (Follow a subject, Subject tour, Follow the action and Cinematic fly-through) share these options:

- **Lead room**: space ahead of moving subjects (0.15 for Follow a subject and Subject tour, 0.1 for the others).
- **Zoom with the action**: zooms in while subjects rest and out while they move fast (0.5 for Follow a subject and Subject tour, 0.3 for the others).
- **Keep in view over hills** (on): raises the camera when a ridge would hide the subject.
- Smooth, spring-like camera movement.

Keyframe files may be a `.json` list of {lon, lat, t?, zoom?, pitch?, bearing?} objects, a `.geojson` of Point features (extras read from properties), or a `.csv`/`.tsv` with lon/lat columns. At least two keyframes are needed. Missing zoom, pitch or bearing values are interpolated between neighbouring keyframes, or fall back to the whole-scene framing and the map's pitch and bearing.

6f. Drawing & Saving

`draw_animated_map` combines the animated trips layer, the terrain tile layer, the view state, the camera and the timeline into a static HTML page (max zoom 15, legend bottom-right). Every animated layer is rebased onto a common timeline starting at its earliest timestamp, so all subjects stay in step. Each layer gets a stable deck.gl id that the page's

animation script uses to find it. `persist_text` writes the page to `animated_map.html`.

Besides the playback bar and Camera panel, the interactive map has deck.gl map buttons (zoom in and out; a compass whose needle follows the heading, where one click sets north up and a second goes flat; reset to the opening view; fullscreen; and a loading spinner), and a legend of subject names in their EarthRanger colours. Terrain and 3D animals are lit by the sun position for the data's date and time. The map buttons are hidden in the video.

7. Video Rendering

Video export is optional and **off by default**. Both video tasks sit in the **Video Export** task group on the form: **Include video?** (`export_video_toggle`), whose **Export video** switch is always visible, and **Video Settings** (`create_animation`), which is only used when the switch is on.

`export_video_toggle` uses `skip_file` (ecoscope-workflows-ext-ste) with the user's **Export video** switch. When the switch is on, it passes the `animated_map.html` path through. When it is off, it returns a skip sentinel, and `create_animation`'s `any_dependency_skipped` condition then skips the whole render. No other task depends on the video, so the map, GeoParquet files and dashboard are unaffected.

> **Note:** The form default comes from an `rjsf-overrides` entry. `skip_file`'s own default is also off, so a config that omits `enabled` skips the video too.

When enabled, `render_video_from_settings` records `animated_map.html` to `animation.mp4`, using the camera from the Camera step (`video_camera`):

- Launches headless Chromium with ANGLE WebGL (GPU when available; SwiftShader software rendering only if `gl` is set to `software`). The spec leaves it on `auto`.
- Reads the timeline span and the animation's natural length. With **Duration** on Auto the video matches the playback **Duration**; otherwise it uses the fixed seconds. Frames = fps × seconds (30 fps; 900 frames by default).
- Computes the whole per-frame camera path in the page from the chosen camera, fitted to the video size and the map's pitch and bearing.
- Splits frames into contiguous chunks across parallel browser pages. `workers: auto` uses half the CPU cores, capped by GPU sharing, free memory and frame count.
- For each frame: sets the view state, renders the animation at that time, waits two animation frames plus any pending tile requests (up to 8 s, then 30 ms settle), and screenshots the map canvas.
- Adds a 1 s hold on the opening frame and a 2 s hold on the closing frame (the completed tracks). The holds reuse captured frames, so they add no capture time.
- Encodes the numbered frames with ffmpeg: libx264, CRF 18, `veryfast` preset, yuv420p, dimensions rounded to even numbers, `+faststart` for web playback.
- Renders are resumable: frames are kept beside the video until it is written, so a failed or interrupted render with the same settings picks up where it stopped.

7a. Video Settings

The user-facing options on the **Video Settings** step are **Duration**, **Overlays** and **Resolution**. The camera is set in the Camera step (section 6d), not here.

Setting	Options
Duration	**Auto** (default) uses the playback length. With Auto off, a fixed number of seconds (default 30).
Overlays	Clock (with a clock position; default top-right), legend, north arrow, scale bar, title and marker labels. All are on by default. The playback bar, Camera panel and map buttons are never shown in the video.

Setting	Options
Resolution	720p (1280×720, default); 1080p; 4K; 720p vertical 9:16 (720×1280); 1080p vertical 9:16 (1080×1920); 1080p square 1:1 (1080×1080); or a custom width and height.

7b. Fixed Encoding Settings

Setting	Value
Frame rate	30 fps
Clip range	Whole timeline (start_frac 0, end_frac 1)
Holds	1 s on the opening frame, 2 s on the closing frame
Frame capture	JPEG, quality 92, device scale factor 1
Encoder	H.264 (libx264), CRF 18, preset veryfast
Workers and WebGL	workers auto, GPU auto
Capture timing	settle 30 ms, tile timeout 8 000 ms, lazy-piece (3D model, filters) timeout 30 000 ms, base map load timeout 60 000 ms
Resume	On: frames kept beside the video until it is written

8. Interactive Dashboard

`create_map_widget_single_view` wraps `animated_map.html` as the **Subject movements** widget. Two `create_files_single_view` steps build the dashboard's **Files** section: `map_files` lists `animated_map.html` and `video_files` lists `animation.mp4` when the video was exported (it is empty otherwise). `gather_dashboard` assembles the map, the Files section, the workflow name and description, the time range and the (empty) groupers. The video is not a dashboard widget; it is offered as a file.

9. Output Files

All outputs are written to `ECOSCOPE_WORKFLOWS_RESULTS`, which can be a local path, a `file://` URL or, on ecoscope-web, a `gs://` folder.

File	Written by	Contents
`relocations_<hash>.geoparquet`	`persist_relocs_geoparquet`	Cleaned GPS fixes
`trajectories_<hash>.geoparquet`	`persist_trajs_geoparquet`	Filtered trajectory segments with renamed columns
`animated_map.html`	`map_urls`	Interactive animated 3D map
`animation.mp4`	`create_animation`	H.264 video of the animation (only when **Export video** is on)

With a `gs://` results folder, the video is rendered on the machine running the workflow (the browser and ffmpeg need local files; the map is downloaded first) and then uploaded to the folder, and the local copy is removed.

10. Workflow Execution Logic

10a. Skip Conditions

Scope	Conditions	Effect
Default (all tasks)	`any_is_empty_df`, `any_dependency_skipped`	A task is skipped if any input dataframe is empty or any upstream task was skipped. An empty EarthRanger result therefore skips the whole chain rather than raising an error.
`export_video_toggle`	Default	Not a skip condition: returns a skip sentinel instead of the HTML path when **Export video** is off.
`create_animation`	`any_dependency_skipped`	Its input is a file path, not a dataframe, so only the upstream-skip check applies. Skipped whenever video export is off.
`animated_map_widget`	`never`	Always runs, so the dashboard always has its map widget.
`video_files`	`never`	Never skipped. With video export off, `create_animation` is skipped and this step falls back to no file, so the dashboard still builds.

10b. Data Flow Summary

Stage	Flow
Ingestion	Observations → relocations → trajectories → renamed columns → GeoParquet
Terrain	Elevation decoder → terrain layer (+ chosen basemap) + terrain sampling
Animation data	EPSG:4326 → trips per subject → densified and draped on terrain → RGBA colours
Framing	Trips envelope → view state (+ pitch, bearing)
Map	Trips animation + trips layer → animated layer; controls → timeline; camera + all → draw_animated_map → HTML
Video (optional)	HTML → Export video toggle → headless Chromium frames (camera from video_camera) → ffmpeg → MP4
Dashboard	HTML → map widget; HTML and video → Files section; all → gather_dashboard

11. Software Versions

Package	Version	Role
ecoscope-platform	==2.25.3	Core runtime and tasks (EarthRanger I/O, preprocessing, CRS, persistence, dashboard)
ecoscope-workflows-ext-custom	0.1.5.*	Map layer definitions and `persist_df_wrapper`
ecoscope-workflows-ext-ste	0.0.32.*	Envelope, view state, hex-to-RGBA and video export toggle tasks

Package	Version	Role
ecoscope-workflows-ext-mep	1.11.0.*	Terrain, trips, animated map, Camera step and video rendering tasks; loads the map widgets (my-deck-widgets v0.21.0) from jsDelivr
pydeck	0.9.2	deck.gl map generation
opentelemetry-sdk	>=1.20.0, <2.0.0	Tracing
wt-compiler	==0.10.1	Compiles `spec.yaml` (development only)

Packages are distributed via the **prefix.dev** conda channels and conda-forge. The runtime environment is managed by **pixi**.